



Advanced Terraform on Azure

Lab 4 – Remote State & Multiple Environments

Expected Duration	1
Teaching Points.....	1
Before You Begin.....	2
Try It Yourself (Optional)	2
Phase 1 – Baseline: Run Starter Code Locally	2
Phase 2 – Create the Remote State Backend	3
Phase 3 – Configure and Migrate to Remote Backend	4
Phase 4 – Introduce Multiple Environments (DEV, TEST, PROD).....	5
Phase 5 – Clean Up	7

Expected Duration

This lab should take 35-45 minutes to complete

Teaching Points

This lab expands your Module 3 configuration by introducing remote state and multi-environment structure. Local state files work only for a single user on a single machine. Remote state enables collaborative, safe Terraform usage for teams.

Key principles:

- terraform.tfvars holds global defaults shared across all environments.
- dev/test/prod var-files override only environment-specific differences.
- Backend state keys isolate environments.

You will:

- Build a storage account to hold Terraform state.
- Migrate your existing Module 3 state into remote storage.
- Create multiple environment configurations (dev, test, prod).

- Learn how Terraform uses state 'keys' to isolate environments.
- Understand why backend values must be literal, not variables.

Before You Begin

Ensure you are familiar with the outcome of Module 3. A copy of the final code revision has been carried forward into the lab4/original and lab4/guided folders. You will start by confirming that the Module 3 solution is functional before migrating to remote state.

- Work only inside the lab4/guided folder.
- Use the lab4/original folder for roll-back if needed
- Use --var-file=<env>.tfvars for environment-specific runs.
- Use terraform.tfvars for global shared values.
- Always specify --var-file explicitly when deploying.
- When switching backend state keys, always run Terraform with -reconfigure.
- Avoid editing the state-backend configuration after creation - it is intentionally minimal.

Try It Yourself (Optional)

Before you begin, think about the following:

- What happens if two people run terraform apply on the same local state file?
- How could a production environment be protected by forcing Terraform to store state remotely?
- How might three state files (dev/test/prod) live inside a single Azure storage container?

Phase 1 – Baseline: Run Starter Code Locally

1. In **lab4/guided**, update **providers.tf** with your **subscription id**.
2. A checker script is provided to identify the statefile currently in use. Run this file now ..

```
cd c:\qatfadvez\lab4\guided
```

```
.\Print-Statefile.ps1
```

```
PS C:\qatfadvez\lab4\guided> .\Print-Statefile.ps1

Terraform backend identity:
  <backend cache not found>

This usually means:
- You have not run 'terraform init' in this folder yet, OR
- A local state file is in use (terraform.tfstate) rather than a remote backend.
```

3. Run ...

```
az login
terraform init
terraform apply --auto-approve
```

4. Re-run Print-Statefile.ps1 ...

```
PS C:\qatfadavaz\lab4\guided> .\Print-Statefile.ps1

Terraform backend identity:
<backend cache not found>

This usually means:
- You have not run 'terraform init' in this folder yet, OR
- A local state file is in use (terraform.tfstate) rather than a remote backend.

Note: A local state file exists here:
terraform.tfstate
```

5. Confirm that **.terraform.lock.hcl** and **terraform.tfstate** exist locally.
6. Use the Azure portal to confirm that resource group, VNet, NSG, and subnets creation succeeds.

You now have a working baseline deployment before migrating to remote state.

Phase 2 – Create the Remote State Backend

7. A course-wide **artifacts** repository exists at the root of your courseware folder. Navigate to the provided **lz-state-backend-bootstrap** folder in **artifacts**. This folder contains only the backend infrastructure code and is an example of a bootstrap configuration.
8. Examine the provided files;
- providers.tf - Azure provider
 - variables.tf - subscription + RG name variables
 - main.tf - resource group, storage account, container
 - outputs.tf - expose RG name, SA name, container name
 - terraform.tfvars - subscription and naming values
9. In **artifacts\lz-state-backend-bootstrap**, update **terraform.tfvars** with your **subscription id** and a random **suffix** for your storage account, which must be uniquely named.

Important !

This remote backend will remain in place for the remainder of this course. Future labs assume its existence and configuration as detailed above. Change **ONLY** your subscription ID and random suffix. Make a note of the suffix chosen as you will require it for future labs. Should the entire lab environment be reset then ensure you recreate this backend first before re-attempting any lab 5 onwards.

10. Run ...

```
cd c:\qatfadavaz\artifacts\lz-state-backend-bootstrap
terraform init
terraform apply --auto-approve
```

In the Azure Portal, verify that the storage account and container are created.

This creates the remote backend prerequisites.

Phase 3 – Configure and Migrate to Remote Backend

11. Return to the **lab4\guided** folder and open **providers.tf**.
12. Add this backend block inside terraform {}, updating the storage account name placeholder with your suffix:

```
backend "azurerm" {  
  resource_group_name = "landing-zone-shared-tfstate-rg"  
  storage_account_name = "advtfstate<your suffix>"  
  container_name      = "tfstate"  
  key = "default.tfstate"  
}
```

Note: Backend blocks cannot use variables or expressions. Values **MUST** be literal.

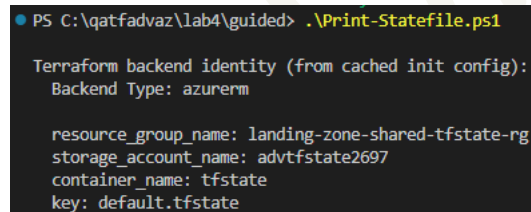
13. Run ...

```
cd c:\qatfadvaz\lab4\guided  
terraform init -migrate-state
```

Enter 'yes' when prompted to migrate

Terraform moves your local terraform.tfstate into the Azure storage container and clears your local terraform.tfstate file.

14. Re-run Print-Statefile.ps1 ...



```
PS C:\qatfadvaz\lab4\guided> .\Print-Statefile.ps1  
  
Terraform backend identity (from cached init config):  
Backend Type: azurerm  
  
resource_group_name: landing-zone-shared-tfstate-rg  
storage_account_name: advtfstate2697  
container_name: tfstate  
key: default.tfstate
```

15. Review the **terraform.tfstate** file in .terraform and note how it references the container name, key, resource group and storage account...

```

"container_name": "tfstate",
"endpoint": null,
"environment": null,
"key": "default.tfstate",
"lookup_blob_endpoint": null,
"metadata_host": null,
"msi_endpoint": null,
"oidc_request_token": null,
"oidc_request_url": null,
"oidc_token": null,
"oidc_token_file_path": null,
"resource_group_name": "landing-zone-shared-tfstate-rg",
"sas_token": null,
"snapshot": null,
"storage_account_name": "advtfstate2697",
"subscription_id": null,

```

You now have a remote, team-safe, backend state. It is using **default.tfstate** and would suffice if a single environment were being maintained.

Phase 4 – Introduce Multiple Environments (DEV, TEST, PROD)

16. For clarity, we will destroy the default deployment ahead of moving to a multi-environment configuration that will create separate state files per environment.

17. Run ...

terraform destroy --auto-approve

18. Each environment requires a named tfvars file...

- Rename terraform.tfvars to **dev.tfvars**
- Duplicate dev.tfvars as **test.tfvars**
- Duplicate dev.tfvars as **prod.tfvars**

19. Update **test.tfvars** ...

```

env = "test"
vnet_address_space = "10.1.0.0/16"

subnet_cidrs ...
  data = "10.1.2.0/24"
  app = "10.1.1.0/24"
  endpoints = "10.1.3.0/24"

```

20. Update **prod.tfvars** ...

```

env = "prod"
vnet_address_space = "10.2.0.0/16"

subnet_cidrs ...
  data = "10.2.2.0/24"

```

```
app = "10.2.1.0/24"
endpoints = "10.2.3.0/24"
```

1. For each environment, reconfigure the backend to use a different state key:

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform apply --var-file=dev.tfvars --auto-approve
```

Check that the local terraform.tfstate file in .terraform is updated to reflect the new key
Run **Print-Statefile.ps1** to confirm

```
terraform init -reconfigure -backend-config="key=test.tfstate"
terraform apply --var-file=test.tfvars --auto-approve
```

Repeat the statefile checks

```
terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform apply --var-file=prod.tfvars --auto-approve
```

Repeat the statefile checks

2. Verify in Azure:

- Three Resource groups exist
- Three VNets exist
- Three NSGs exist
- Each environment has its own three subnets
- All NSGs use the same rule

3. Verify in Azure Storage that alongside the now redundant default.tfstate file there are environment specific state files...

- dev.tfstate
- test.tfstate
- prod.tfstate

Takeaway

You now have a multi-environment Terraform pattern.

Terraform always stores backend *configuration* locally in .terraform/terraform.tfstate, so always reconfigure when switching. To prevent accidental cross-environment drifts, follow this sequence when switching between environments:

```
terraform init -reconfigure -backend-config="key=..."
terraform plan/apply/destroy -var-file=...
```

Phase 5 – Clean Up

4. Run ...

```
cd c:\qatfadvaz\lab4\guided
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform destroy --var-file=dev.tfvars --auto-approve
terraform init -reconfigure -backend-config="key=test.tfstate"
terraform destroy --var-file=test.tfvars --auto-approve
terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform destroy --var-file=prod.tfvars --auto-approve
```

Use the portal to verify all lab resources other than the remote backend have been destroyed.

***** Congratulations, you have completed this lab. *****

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026